RICE UNIVERSITY

T3 · Parallel training systems

Write data-parallel gradient averaging and check it against DDP; complete tensor and pipeline exercises; justify a 3D layout.

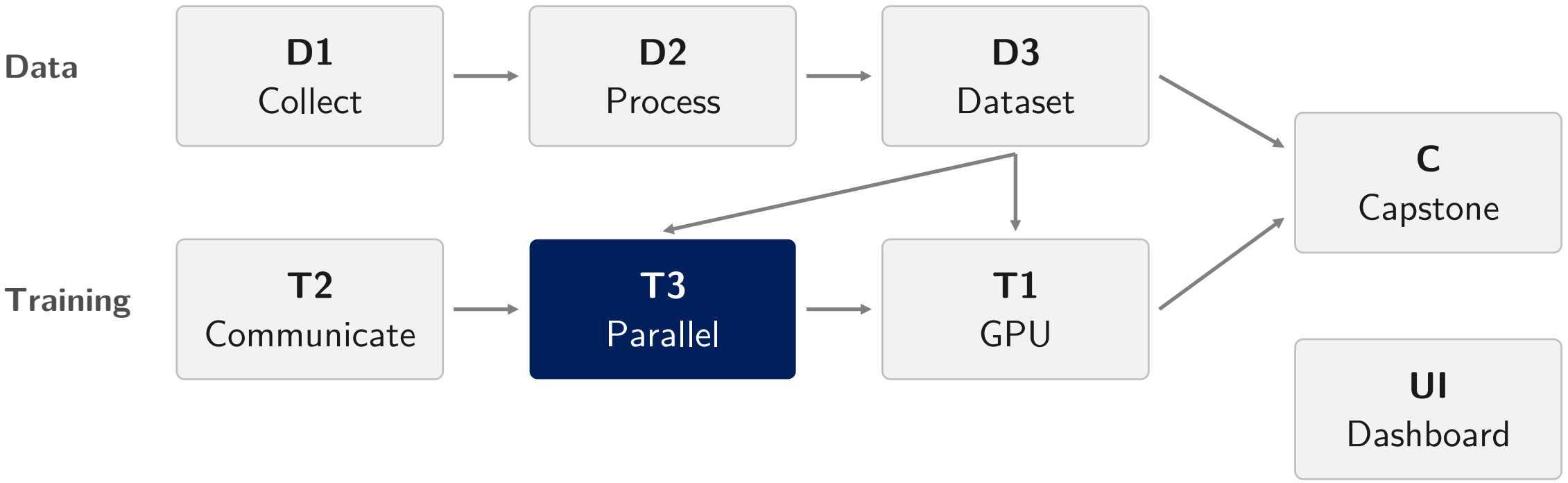
COMP 329 course staff

Rice University

Spring 2027 · Assignment T3

Release week 10 · Checkpoint week 11 · Due week 12 · 7% of grade

Where this assignment fits



In our pipeline

This assignment: `runs/inputs/bpe` → **T3** → `assignment-result.json`

What you will build, and why

Train on your D3 data with gradients averaged across GPUs, then choose how to split a large model across 64 GPUs and defend the choice.

1 **Data parallelism**

The same model on every GPU; one all-reduce per step.

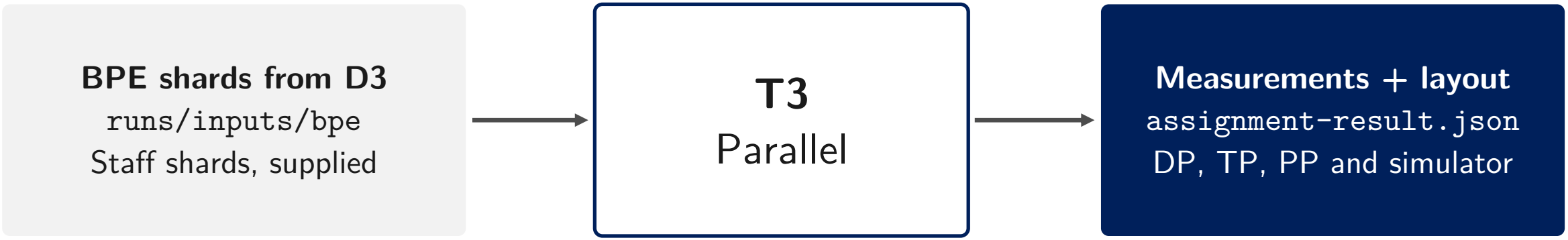
2 **Tensor and pipeline**

Split matrices or layers when one GPU cannot hold the model.

3 **Layouts are trade-offs**

Memory, communication and bubbles pull in different directions.

What goes in, what comes out



Files you produce

`dp_check`

One update vs. one process and DDP

`dp_training`

Step time, all-reduce time, tokens/s per world size

`pipeline_parallel`

Bubble fraction for 1, 2, 4, 8 microbatches

Average gradients over the global batch

```
bucket = torch.cat(
  [p.grad.reshape(-1) for p in params])
bucket *= local / global_
dist.all_reduce(bucket)  # SUM
# copy slices back into p.grad
```

1 **Uneven batches**

17 examples on 2 ranks: weights 9/17 and 8/17, not 1/2.

2 **One bucket**

One large message pays α once.

3 **Check it**

Match one process and DDP to 1e-6.

4 **Batch convention**

Fixed global batch, split across ranks.

Tensor and pipeline parallelism

1

Column split

Each rank: $\tanh(x \cdot W1[:, cols] + b1)$.

2

Row split

Partial $= h \cdot W2[rows, :]$.

3

All-reduce

Sum partials; add b2 once.

4

Pipeline

Layers in stages; microbatches flow through.

5

Bubble

Idle fraction $(P - 1)/(M + P - 1)$.

Key idea

GPipe: all forwards, then all backwards. More microbatches shrink the bubble but keep more activations.

Choose a DP × TP × PP layout

Rank layout

`rank = t + tp·(d + dp·p)`

TP fastest: keep it inside one server.

Memory

`weights 2 + grads 2 + Adam 12 bytes/param`

ZeRO shards these over DP.

Time

`compute / (1 − bubble) + TP + PP + DP comm`

From the supplied simulator.

Key idea

Justify one layout for 64 GPUs with 80 GB each: why that TP, PP and DP, and what the simulator leaves out.

What you write, what you get

You write

`layout, average_gradients`

`starter/parallel.py` · checkpoint

`tp_mlp_forward, pipeline_schedule`

`starter/parallel.py` · guided

Provided

- Model, optimizer, loader
- Launcher, DP training loop, DDP comparison
- DeepMLP toy model; TP/PP runners
- 3D layout simulator

Ground rules

Keep the signatures · don't edit `starter/pipeline_lab` · run through `course.py` or `lab.py`.

Build it in this order

1	2	3	4	5
Layout	**DP update**	**Tensor parallel**	**Pipeline**	**Measure + choose**
Coordinates and groups.	Weighted bucket, one all-reduce.	Column then row split.	GPipe timetable.	DP scaling; simulator layout.
`check t3 --checkpoint`	`same (checkpoint)`	`check t3`	`check t3 --systems`	`lab.py t3`

Check

Done when: All 9 tests pass (4 checkpoint tests; NCCL on two GPUs); one layout is justified under a fixed budget.

If something goes wrong

DP error ≈ 1e-2 with 9/8 split
Check: Weight by local/global, not 1/world.

A rank hangs
Check: Same number of all-reduces on every rank.

TP output too large
Check: b2 added before the sum.

Schedule rejected
Check: Forward slot m + s; backward flows back.

2 GPUs slower than 1
Check: Expected for a tiny model; explain with α–β.

Memory claims without units
Check: Bytes per parameter, per GPU.

The performance study

What limits multi-GPU speedup, and which layout fits the budget?

Predict first

DP step $\approx$ compute/G + all-reduce of $4 \cdot P$ bytes. For this tiny model communication dominates, so expect efficiency well below 1.

World size 1 / 2
ours vs. DDP

Microbatches 1 / 2 / 4 / 8
2 stages

ZeRO 0 / 1 / 3
simulator, 64 GPUs

How to measure: State the batch convention. Report scaling efficiency, communication volume and bubble fraction.

How you are graded

20	40	16	24
Checkpoint	Final correctness	Performance	Analysis

Correctness: layout 6 · DP averaging and training 12 · tensor parallel 6 · pipeline 6 · walkthrough 10

Check

Done when: All 9 tests pass (4 checkpoint tests; NCCL on two GPUs); one layout is justified under a fixed budget.

Your first ten minutes

```
$ bash setup.sh
$ source .venv/bin/activate
$ python course.py preflight
$ python course.py check t3 --checkpoint
$ python lab.py t3 --input runs/inputs/bpe \
    --output runs/t3-first --steps 40
$ python 06_t3_scale_out/layout_sim.py \
    --gpus 64
```

Your folder

`06_t3_scale_out/`

Handout:
`06_t3_scale_out/README.md`